ПРАКТИКУМ З ПРОГРАМУВАННЯ №4

Войлов Кирилл ІПЗ 24 - 1/9


```
#define _USE_MATH_DEFINES

#include <iostream>

#include <cmath>

#include <limits>



using namespace std;



typedef struct {

    double re;
```

```
    double im;

} complex;



complex complex_div(complex num, complex den) {

    complex result;

    double denom = den.re * den.re + den.im * den.im;

    result.re = (num.re * den.re + num.im * den.im) / denom;

    result.im = (num.im * den.re - num.re * den.im) / denom;

    return result;

}



void complex_print(complex z) {

    if (z.im >= 0)
```

```
        cout << z.re << " + i * " << z.im;

    else

        cout << z.re << " + i * " << z.im;

}



double read_double(const string& prompt) {

    double val;

    while (true) {

        cout << prompt;

        if (cin >> val) return val;

        cout << "Invalid input. Try again." << endl;

        cin.clear();

        cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
    }

}



int main() {

    int circuit;



    cout << "Enter circuit number (1-4): ";

    while (!(cin >> circuit) || circuit < 1 || circuit > 4) {

        cout << "Invalid input. Enter circuit number (1-4): ";

        cin.clear();

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

    }
```

```
double R1 = 0.0, R2 = 0.0, L, C, f_min, f_max, df;



bool need_R = (circuit == 1 || circuit == 2);

bool need_R1 = (circuit == 3 || circuit == 4);

bool need_R2 = (circuit == 3 || circuit == 4);




if (need_R) {

   do {

      R1 = read_double("Enter R(Ohm): ");

      if (R1 <= 0) cout << "R must be positive. Try again." << endl;

   } while (R1 <= 0);

}

if (need_R1) {
```

```
    do {
        R1 = read_double("Enter R1(Ohm): ");
        if (R1 <= 0) cout << "R1 must be positive. Try again." << endl;
    } while (R1 <= 0);
}

if (need_R2) {
    do {
        R2 = read_double("Enter R2(Ohm): ");
        if (R2 <= 0) cout << "R2 must be positive. Try again." << endl;
    } while (R2 <= 0);
}

do {
```

```
    L = read_double("Enter L(mHn): ");

    if (L <= 0) cout << "L must be positive. Try again." << endl;

} while (L <= 0);



do {

    C = read_double("Enter C(mcF): ");

    if (C <= 0) cout << "C must be positive. Try again." << endl;

} while (C <= 0);



do {

    f_min = read_double("Enter F_min: ");

    if (f_min <= 0) cout << "F_min must be positive. Try again." << endl;

} while (f_min <= 0);
```

```
do {
    f_max = read_double("Enter F_max: ");
    if (f_max <= 0) { cout << "F_max must be positive. Try again." << endl; continue; }
    if (f_max < f_min) { cout << "F_max must be >= F_min. Try again." << endl; continue; }
    break;
} while (true);

do {
    df = read_double("Enter step: ");
    if (df <= 0) cout << "Step must be positive. Try again." << endl;
} while (df <= 0);
```

```
double f0 = 1.0 / (2.0 * M_PI * sqrt(L * 1e-3 * C * 1e-6));

cout << "resonance frequency: " << f0 << endl;



for (double f = f_min; f <= f_max + 1e-9; f += df) {

    double omega = 2.0 * M_PI * f;

    double X_C = 1.0 / (omega * C * 1e-6);

    double X_L = omega * L * 1e-3;



    complex num, den, Z;



    switch (circuit) {

    case 1:

        num.re = (L * 1e-3) / (C * 1e-6);
```

```
    num.im = -R1 * X_C;

    den.re = R1;

    den.im = X_L - X_C;

    Z = complex_div(num, den);

    break;

case 2:

    num.re = (L * 1e-3) / (C * 1e-6);

    num.im = R1 * X_C;

    den.re = R1;

    den.im = X_L - X_C;

    Z = complex_div(num, den);

    break;

case 3:
```

```
    num.re = R1 * R2;

    num.im = R1 * (X_L - X_C);

    den.re = R1 + R2;

    den.im = X_L - X_C;

    Z = complex_div(num, den);

    break;

case 4:

    num.re = R1 * R2 + (L * 1e-3) / (C * 1e-6);

    num.im = X_L * R1 - R2 * X_C;

    den.re = R1 + R2;

    den.im = X_L - X_C;

    Z = complex_div(num, den);

    break;
```

```
        default:

            num.re = 0; num.im = 0;

            Z = num;

        }



        cout << "f=" << f << "\tZ= ";

        complex_print(Z);

        cout << endl;

    }



    return 0;

}
```